SUMO Simulation: Lateral Velocity of the Ego, the Adaptive Policy and Identical Runs

Sep 25 – Oct 9

1 Lateral velocity of the ego: old code vs new code

1.1 The issue

- The DIPF potential commands a lateral acceleration $a_y = -\partial U/\partial y$ (up to 3 m/s^2).
- In the **old code** the ego barely moves sideways: lane changes are rare and very slow.
- Cause: the lateral velocity v_y is read back from SUMO every step, and SUMO always returns 0.
- The **new code** keeps v_y as controller state, so the push builds up into real lateral motion.

1.2 Old code: v_y is re-read from SUMO every step

Files: `v2/run1/dipf_controller.py`, `Pure_DIPF_codes.py`.

```
vx = traci.vehicle.getSpeed(EGO_ID)
vy = traci.vehicle.getLateralSpeed(EGO_ID)        # always 0 for the ego

ay = clamp(-dUy, -AY_MAX, AY_MAX)
vy_apply = clamp(vy + ay*dt, -VY_MAX, VY_MAX)     # computed, never stored
y_new = y + vy*dt + 0.5*ay*dt**2                  # vy = 0:  only 0.5*ay*dt**2 is left
traci.vehicle.moveToXY(EGO_ID, ..., x_new, y_new, ...)
```

- The longitudinal speed works: `setSpeed()` stores it in SUMO and `getSpeed()` reads it back.
- There is no equivalent for the lateral speed.

1.3 Why SUMO returns $v_y = 0$

- Sublane model (`-lateral-resolution 0.25`, lane-change model SL2015): lanes are split into 0.25 m sublanes, vehicles can sit at any lateral position and move sideways gradually.
- `getLateralSpeed()` returns the lateral speed **commanded by SUMO's lane-change model**, not the measured motion.
- The ego's lane-change model is disabled (`setLaneChangeMode(ego, 0)`) and the ego is placed with `moveToXY` (a teleport) $\Rightarrow$ no manoeuvre $\Rightarrow$ $v_y = 0$.
- Other vehicles are moved by the lane-change model, so their lateral speed is correct.

Check in SUMO. The ego was moved 0.2 m sideways every step (2 m/s) with `moveToXY` (scenario s05003, SUMO 1.27.1):

step	ego moved sideways	ego `getLateralSpeed`	other vehicles (max)
1	+0.20 m	0.00 m/s	0.10 m/s
4	+0.20 m	0.00 m/s	0.40 m/s
8	+0.20 m	0.00 m/s	0.80 m/s
10	+0.20 m	0.00 m/s	0.50 m/s

1.4 Effect on the motion

Constant lateral push $a_y = 3$ m/s^2, $v_x = 15$ m/s:

time	old code: y	new code: y
0.5 s	0.08 m	0.38 m
1.0 s	0.15 m	1.50 m
1.5 s	0.23 m	3.00 m
3.0 s	0.45 m	7.50 m

- Old: $\Delta y = \frac{1}{2} a_y \Delta t^2 \leq 0.015$ m per step, i.e. about 0.15 m/s of drift (lane width 3.5 m).
- New: v_y builds up to its 3 m/s cap in 1 s; a lane change takes about 1.5 s.

Why the old code still shows some lateral movement

- The $\frac{1}{2} a_y \Delta t^2$ term accumulates under a sustained push (e.g. stuck behind a slow vehicle): 1.5 m after 10 s at 3 m/s^2.
- Fallback in `safe_move_ego`: if `moveToXY` fails, the ego is placed at the nearest lane centre (a jump of up to 1.75 m).
- y is clamped to the safe road band, so the ego is pushed back inside instead of crashing at the edge.

1.5 New code: v_x, v_y kept by the controller

File: `v14/*/dipf_controller.py` (`ego_control_step`).

```
# once, when the ego appears
ego_control_step.vx = traci.vehicle.getSpeed(EGO_ID)
ego_control_step.vy = 0.0

# every step
vx, vy = ego_control_step.vx, ego_control_step.vy
vy_apply = clamp(vy + ay*dt, -VY_MAX, VY_MAX)
vy_apply = clamp(vy_apply, -vx*tan(15 deg), vx*tan(15 deg))   # heading limit
y_new = y + 0.5*(vy + vy_apply)*dt                            # trapezoidal step
traci.vehicle.moveToXY(EGO_ID, ..., x_new, y_new, angle=heading)
ego_control_step.vy = vy_apply                                 # stored for the next step
```

1.6 Summary

	old code	new code
lateral velocity	re-read from SUMO, always 0	controller state, integrated
position	front bumper used as body centre	body centre
heading	chosen by SUMO	from (v_x, v_y), passed to `moveToXY`
v_y vs v_x	no link	$\lvert v_y\rvert \le v_x \tan 15°$
road edge	position clamped to the road	leaving the road = collision
`moveToXY` failure	snap to the nearest lane centre	reported as an error

2 CMA-ES: the optimiser used in v14

Both the fixed base set (`v14/base`) and the adaptive policy (`v14/adaptive`) are trained with the Covariance Matrix Adaptation Evolution Strategy (CMA-ES, N. Hansen, *The CMA Evolution Strategy: A Tutorial*, 2016). It is a derivative-free optimiser: it only needs a score for each candidate, which suits J (a closed-loop SUMO success rate, noisy and without a gradient).

2.1 Idea

CMA-ES keeps a Gaussian **search distribution** $\mathcal{N}(m, \sigma^2 C)$ over the search vector $x \in \mathbb{R}^N$:

- m (mean): the current best guess, *where* to search;
- σ (step size): *how far* to search around m;
- C (covariance, $N \times N$): the *shape* of the search, stretched along directions that worked.

Each generation samples λ candidates, scores them, moves m to a weighted average of the best μ, and adapts σ and C from the step just taken. Two **evolution paths** remember the recent steps: p_σ (for the step size) and p_c (for the covariance).

2.2 Constants

Only λ (population size) and the start values m_0, σ_0 are chosen; everything else follows from N and λ (standard defaults):

$$\mu = \lfloor \lambda/2 \rfloor, \qquad w_i = \frac{\ln(\mu + \frac{1}{2}) - \ln i}{\sum_{j=1}^{\mu} \big(\ln(\mu + \frac{1}{2}) - \ln j\big)}, \quad i = 1, \dots, \mu,$$

$$\mu_{\text{eff}} = \Big(\sum_{i=1}^{\mu} w_i^2\Big)^{-1}, \qquad \chi_N = \sqrt{N}\Big(1 - \tfrac{1}{4N} + \tfrac{1}{21N^2}\Big) \approx \mathbb{E}\|\mathcal{N}(0, I)\|,$$

$$c_\sigma = \frac{\mu_{\text{eff}} + 2}{N + \mu_{\text{eff}} + 5}, \qquad d_\sigma = 1 + 2\max\Big(0, \sqrt{\tfrac{\mu_{\text{eff}} - 1}{N+1}} - 1\Big) + c_\sigma,$$

$$c_c = \frac{4 + \mu_{\text{eff}}/N}{N + 4 + 2\mu_{\text{eff}}/N}, \qquad c_1 = \frac{2}{(N + 1.3)^2 + \mu_{\text{eff}}},$$

$$c_\mu = \min\Big(1 - c_1,\ \frac{2(\mu_{\text{eff}} - 2 + 1/\mu_{\text{eff}})}{(N+2)^2 + \mu_{\text{eff}}}\Big).$$

	`v14/base` ($N = 7$)	`v14/adaptive` ($N = 36$)
λ, μ	15, 7	15, 7
$w_1 \dots w_7$	0.361, 0.237, 0.164, 0.113, 0.073, 0.040, 0.012	
μ_{eff}	4.29	4.29
c_σ, d_σ	0.386, 1.386	0.139, 1.139
c_c	0.377	0.102
c_1, c_μ	0.0273, 0.0591	0.0014, 0.0035
χ_N	2.554	5.959

With $N = 36$ the covariance learns slowly ($c_1 + c_\mu \approx 0.005$ per generation): in 32 generations the adaptive search is mainly driven by m and σ, while the 7-dimensional base search also reshapes C noticeably ($c_1 + c_\mu \approx 0.09$).

2.3 Algorithm

Initialise: $m = m_0$, $\sigma = \sigma_0$, $C = I$, $p_\sigma = p_c = 0$, $g = 0$.
Repeat for every generation $g = 0, 1, 2, \ldots$

1. **Sample.** Eigendecomposition $C = BD^2B^\top$ (B: eigenvectors, D: square roots of the eigenvalues). For $k = 1, \ldots, \lambda$:
$$z_k \sim \mathcal{N}(0, I), \qquad y_k = BDz_k \sim \mathcal{N}(0, C), \qquad x_k = m + \sigma\, y_k.$$
2. **Evaluate.** Score every candidate, $f_k = f(x_k)$ (lower is better), and sort: $f(x_{1:\lambda}) \le f(x_{2:\lambda}) \le \ldots$, where $x_{i:\lambda}$ is the i-th best.
3. **Move the mean.** Weighted average of the μ best steps:
$$\bar{y}_w = \sum_{i=1}^{\mu} w_i\, y_{i:\lambda}, \qquad m \leftarrow m + \sigma\, \bar{y}_w.$$
4. **Step-size path** (in whitened coordinates, so it is independent of the shape C):
$$p_\sigma \leftarrow (1 - c_\sigma)\, p_\sigma + \sqrt{c_\sigma(2 - c_\sigma)\, \mu_{\text{eff}}}\; C^{-1/2}\, \bar{y}_w, \qquad C^{-1/2} = BD^{-1}B^\top.$$
5. **Stall indicator.** $h_\sigma = 1$ if $\dfrac{\|p_\sigma\|}{\sqrt{1 - (1 - c_\sigma)^{2(g+1)}}} < \left(1.4 + \frac{2}{N+1}\right) \chi_N$, else 0 (stops p_c from growing while σ is still increasing quickly).
6. **Covariance path:**
$$p_c \leftarrow (1 - c_c)\, p_c + h_\sigma \sqrt{c_c(2 - c_c)\, \mu_{\text{eff}}}\; \bar{y}_w.$$
7. **Covariance update** (rank-one from the path, rank-μ from this generation's best steps):
$$C \leftarrow (1 - c_1 - c_\mu)\, C + c_1 \left(p_c p_c^\top + (1 - h_\sigma)\, c_c(2 - c_c)\, C\right) + c_\mu \sum_{i=1}^{\mu} w_i\, y_{i:\lambda}\, y_{i:\lambda}^\top, \qquad C \leftarrow \tfrac{1}{2}(C + C^\top).$$
8. **Step-size update:**
$$\sigma \leftarrow \sigma \exp\Big(\frac{c_\sigma}{d_\sigma}\Big(\frac{\|p_\sigma\|}{\chi_N} - 1\Big)\Big).$$
9. $g \leftarrow g + 1$.

What each update does.

- **Mean:** only the better half of the samples pulls m, the best one most ($w_1 = 0.36$). Because m is an average of several candidates, one lucky score moves it little.
- **Step size:** if successive mean steps point the same way, $\|p_\sigma\| > \chi_N$ and σ grows (the search is too timid); if they cancel, $\|p_\sigma\| < \chi_N$ and σ shrinks (the search is circling an optimum). For random steps $\|p_\sigma\| \approx \chi_N$ and σ stays.
- **Covariance:** the rank-one term stretches C along the direction the mean has been moving over several generations (p_c); the rank-μ term stretches it along this generation's successful steps. Correlated parameters (good sets have, e.g., more λ_{obs} *and* less τ_y) are therefore searched along their diagonal.

File: `v14/*/common.py` (`class CMAES`: `ask` = step 1, `tell` = steps 3–9).

2.4 CMA-ES in v14

	`v14/base`	`v14/adaptive`
search vector x	7 DIPF parameters, log-scaled to $u \in [0,1]^7$ (below)	$W \in \mathbb{R}^{4\times 8}$, $b \in \mathbb{R}^4$ (36 numbers), exponents of $p = p_0 e^z$
start m_0	v12 library set 2	$W = 0$, $b = 0$ (= the trained base)
σ_0	0.15 (15 % of each log range)	0.5 ($e^{0.5} = \times 1.65$ per unit feature)
cost f	$-J + 0.1 \cdot$distance of u outside $[0,1]^7$	$-J$
bounds	u clipped to $[0,1]$ when simulated, penalty above steers m back	sign constraints by reflection, $w \leftarrow \lvert w \rvert$
budget	$\lambda = 15$, 32 generations $\Rightarrow$ 480 candidates (same for both)	
one evaluation	J on 100 train scenarios $\times$ 2 futures = 200 SUMO runs	

with $J =$ success rate $+ 0.1 \cdot$ mean progress (progress $= 1$ for a success), and for the base

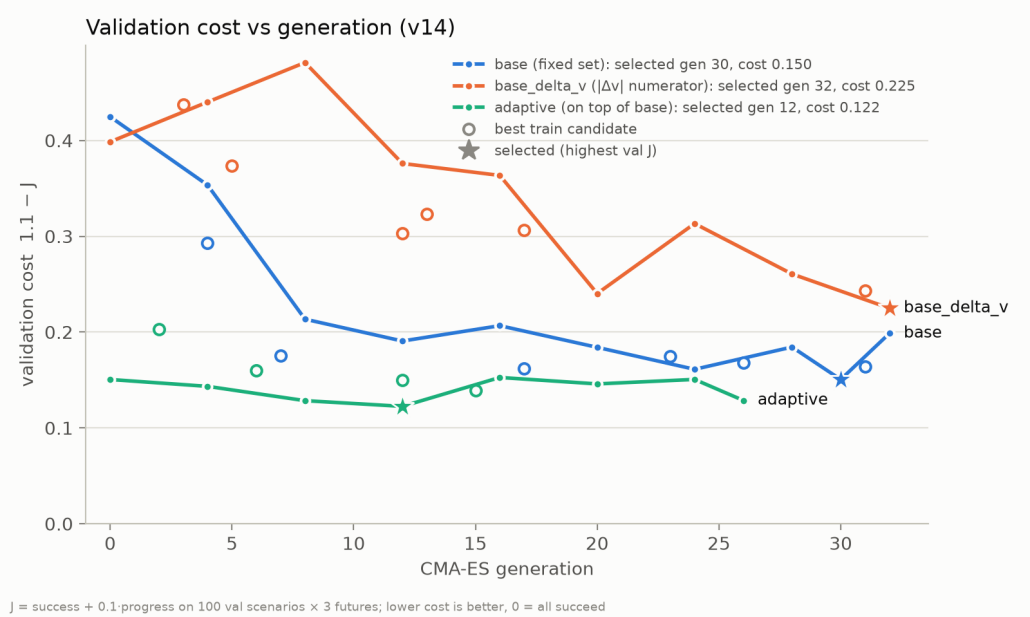
$$u_k = \frac{\log_{10} p_k - \log_{10} lo_k}{\log_{10} hi_k - \log_{10} lo_k}, \qquad [lo_k, hi_k] = \text{search range of parameter } k,$$

so one unit of u spans the whole log range and σ means the same for every parameter.

Validation and selection. CMA-ES only sees the training runs. Every 4 generations two points are scored on validation (100 scenarios $\times$ 3 futures):

- the **mean** m (a new point, never one of the sampled candidates);
- the **best train candidate** so far, if a new one appeared.

The final set is the validated point (start set included) with the highest validation J; ties go to the earlier generation. In all three v14 runs this was the mean m: base generation 30 (validation $J = 0.950$), `base_delta_v` generation 32 (0.875), adaptive generation 12 (0.978). The best train candidates, picked as the highest of many noisy training scores, dropped back on validation (e.g. adaptive generation 15: train $J = 0.998$, validation 0.961).



3 The adaptive policy algorithm (v14)

The DIPF controller itself is unchanged: every acceleration is still the clamped negative gradient of the potential. The adaptive policy only **retunes four of the potential's parameters every** 0.1 s from the traffic around the ego. It never outputs an acceleration, so a bad choice can reshape the field but cannot command an arbitrary manoeuvre.

3.1 Algorithm

Every control step, just before the potential is evaluated:

$$
\begin{aligned}
f_t &= \text{features(80-value traffic encoding)} \in [0,1]^9 \\
z_t^* &= \text{clip}(W f_t + b,\ -\ln 10,\ +\ln 10) && W \in \mathbb{R}^{4\times 9},\ b \in \mathbb{R}^4 \ \Rightarrow\ 40 \text{ numbers} \\
z_t &= z_{t-1} + \tfrac{\Delta t}{T}\,(z_t^* - z_{t-1}) && \Delta t = 0.1 \text{ s},\ T = 1 \text{ s};\ z_0 = z_0^* \\
p_t &= \text{clamp}(p_0 \cdot e^{z_t},\ \text{limits}) && p = (\lambda_{obs},\ \tau_y,\ \alpha,\ \text{obs_radius_ahead})
\end{aligned}
$$

- p_0 = the trained fixed set from `v14/base` (CMA-ES over all 7 parameters). $W = 0, b = 0$ reproduces it exactly, so any gain comes from adapting.
- e^z: multiplicative and always positive; the clip keeps each parameter within $\times 0.1 \cdots \times 10$ of p_0.
- Smoothing: the exponent moves 10 % of the way to its target each step, so a parameter follows a sudden change within about 1 s instead of jumping.
- The other parameters (s_x, δ_{lane}, y_{margin}, $\tau_x = 1$) stay at p_0.
- Sign constraint: the weights from `ttc_front`, `ttc_side_front`, `drac_front` to λ_{obs} are ≥ 0 (repulsion may only grow with danger).

File: `v14/adaptive/dipf_controller.py` (`adapt_parameters`), `policy.py`.

```
enc = encode((x, y, vx, vy), nearby, road_bounds_y(), EGO_LENGTH, EGO_WIDTH)
f = pol.features(enc)                                    # 9 features in [0, 1]
z_target = clip(W @ f + b, -ln10, ln10)
z = z_target if z is None else z + min(1, dt/T) * (z_target - z)   # smoothing
for k, name in enumerate(params):                        # lambda_obs, tau_y, alpha, radius
    value[name] = clamp(base[name] * exp(z[k]), *limits[name])
# ... then the unchanged DIPF step: U, grad U, a = clamp(-grad U), moveToXY
```

3.2 Inputs: nine features

All in $[0, 1]$, 0 = nothing there / far away, and unchanged when left and right are swapped (like the controller).

feature	definition	large when
`ttc_front`	$1 - e^{-1/\text{TTC}}$, vehicle in the ego's path ahead	closing in fast ahead
`ttc_side_front`	same, worst of front-left / front-right	closing on a diagonal vehicle
`ttc_rear_side`	same, worst of rear, rear-left/right, alongside	closing from behind/side
`drac_front`	$1 - e^{-\text{DRAC}/3}$, braking for the car ahead	hard braking needed
`gap_front`	$e^{-\text{gap}/20\,\text{m}}$	vehicle close ahead
`gap_side_front`	same, closest of front-left / front-right	vehicle close diagonally ahead
`tight_lateral`	$e^{-\text{room}/1\,\text{m}}$ (road edge or side gap)	no room to swerve
`speed`	$v_x/34$ m/s	driving fast
`density`	$1 - e^{-n/10}$, n = vehicles within ± 50 m	crowded

3.3 Training

- No target value of p_t exists, so the 40 numbers are trained directly on closed-loop outcome: J = success rate + $0.1 \cdot$ mean progress, takeover ($x \approx 100$ m) to $x = 400$ m.
- CMA-ES from $W = 0, b = 0$: 30 generations $\times$ 15 candidates, each scored on the same 100 train scenarios $\times$ 2 futures (same budget as the fixed base: 450 candidates).
- Every 4 generations the search mean is scored on validation (100×3 runs); the best one is kept (generation 12, validation success 0.730 vs 0.647 for the fixed set).

3.4 What it learned

- λ_{obs} (repulsion): rises sharply with a vehicle closing from behind/side (`ttc_rear_side` +2.0) or diagonally ahead (+1.85), and with tight lateral room (+1.3); falls behind a close but non-closing car (`gap_front` −1.7), so the ego follows calmly.
- τ_y (lateral reach): grows with speed (+1.5), shrinks when closing in ahead (−1.3).
- α: pinned at its 0.01 floor in every step, so it does not really adapt.
- Look-ahead radius: about $\times$1.8 of the fixed set ($\approx$ 174 m vs 97 m), shorter in dense traffic.

3.5 Results

Test: 300 fresh scenarios $\times$ 3 futures = 900 paired runs (identical traffic for every controller, Section 5).

controller	success	progress
v12 library set 2 (v13 base)	0.588	0.750
trained fixed set (`v14/base`)	0.646	0.780
adaptive offsets only ($W = 0$)	0.621	0.751
adaptive policy (`v14/adaptive`)	**0.714**	**0.812**

- Adaptive vs trained fixed set: $+6.9 \pm 2.2$ points (only adaptive succeeded 182, only fixed 120; McNemar $p = 0.0004$).
- The gain comes from adapting: the offsets alone are no better than the fixed set (−2.4, $p = 0.21$).

4 Constraints on the motion and on the potential (v14)

The controller has two kinds of limits: **hard limits on the motion** (applied after the gradient, they override the potential) and **built-in margins in the potential** (they shape the field, so the gradient already keeps the ego away from the limits). Values are for `v14/base` and `v14/adaptive` (`config.yaml`, `base_best.json`); ego body $L \times W = 4.583 \times 1.748$ m.

4.1 Motion limits: acceleration, velocity, jerk

quantity	lower	upper	applied to
longitudinal acceleration a_x	-9.0 m/s^2 (braking)	$+3.2$ m/s^2	$-\partial U/\partial x$
lateral acceleration a_y	-3.0 m/s^2	$+3.0$ m/s^2	$-\partial U/\partial y$
longitudinal speed v_x	0 (no reversing)	34 m/s	$v_x + a_x \Delta t$
lateral speed v_y	-3.0 m/s	$+3.0$ m/s	$v_y + a_y \Delta t$
heading (see below)	$\lvert v_y \rvert \le v_x \tan 15^\circ$		after the v_y clamp
jerk $\dot{a}_x, \dot{a}_y$	**none**		–

- The braking limit 9 m/s^2 is SUMO's emergency deceleration; the throttle limit is asymmetric (3.2). The flag `ego.limit_acceleration: false` removes the acceleration clamp (speed limits stay).
- The acceleration is clamped **per axis** (box, not a friction circle): the ego may brake at 9 and swerve at 3 m/s^2 at the same time.
- **No jerk limit** (since v9): the acceleration is the clamped gradient of the current step and may jump to any value in the box between two steps. The largest possible jump in one $\Delta t = 0.1$ s step is therefore $|\Delta a_x|/\Delta t = (3.2 + 9.0)/0.1 = 122$ m/s^3 and $|\Delta a_y|/\Delta t = 6/0.1 = 60$ m/s^3 (comfort limits are typically 1–2 m/s^3). The previous acceleration is stored (`prev_ax`, `prev_ay`) but not used.
- The only smoothing in the loop is on the **parameters** (adaptive policy, Section 4.4), not on the acceleration.

File: `v14/*/dipf_controller.py` (`ego_control_step`).

```
ax = clamp(-dUx, -EGO_D_MAX, EGO_A_MAX)            # [-9.0, +3.2]
ay = clamp(-dUy, -EGO_AY_MAX, EGO_AY_MAX)          # [-3.0, +3.0]
# No jerk limit (v9): the commanded acceleration is the clamped gradient.   # no rate limit
vx_apply = clamp(vx + ax*dt, 0.0, EGO_MAX_SPEED)   # [0, 34]
vy_apply = clamp(vy + ay*dt, -EGO_VY_MAX, EGO_VY_MAX)
vy_cap   = vx_apply * TAN_MAX_HEADING               # tan(15 deg)
vy_apply = clamp(vy_apply, -vy_cap, vy_cap)
```

4.2 Angle constraints

Heading limit. The ego's heading follows its velocity, $\theta = \arctan(v_y/v_x)$, and is passed to `moveToXY`. It is capped at $\theta_{max} = 15°$ by limiting the lateral speed:

$$|v_y| \le v_x \tan 15° = 0.268\, v_x.$$

- Ties lateral to forward motion: no sideways sliding at low speed; at standstill $v_y = 0$ (the heading keeps its last value).
- The 3 m/s lateral cap is the binding one only above $v_x = 3/0.268 = 11.2$ m/s; below that the heading cap is stricter (e.g. $v_x = 5$ m/s $\Rightarrow |v_y| \le 1.34$ m/s).

Angled body in the lane potential. A turned ego is wider across the road. The lane walls use its lateral half-extent at the current heading:

$$h(\theta) = \tfrac{L}{2}|\sin\theta| + \tfrac{W}{2}|\cos\theta|$$

θ	0°	5°	10°	15° (max)
$h(\theta)$	0.874 m	1.070 m	1.259 m	1.437 m

So a lane change towards the edge moves the wall up to 0.56 m inwards. Note: the hard road-departure check (`ego_within_road`) uses the unturned $W/2$, so the potential is the more conservative of the two.

Region angle in the obstacle potential. k' switches between its longitudinal and lateral form along the diagonal of the combined box ($w = (W_{ego} + w_{eff})/2$, $l = (L_{ego} + l_{eff})/2$):

$$k' = \begin{cases} (1 - \frac{l}{|\Delta x|})\sqrt{\left(\frac{\Delta x}{\tau_x}\right)^2 + \left(\frac{\Delta y}{\tau_y}\right)^2} & |\Delta y| \le \frac{w}{l}|\Delta x| \quad \text{(ahead / behind)} \\ (1 - \frac{w}{|\Delta y|})\sqrt{\cdots} & \text{otherwise} \quad \text{(beside)} \end{cases}$$

i.e. the boundary is the corner direction $\arctan(w/l)$ from the road axis: 21.2° (bike), 23.8° (auto), 21.9° (car), 15.0° (bus, truck). In each region the singular wall sits where the two bodies touch on that axis.

4.3 Margins built into the potentials

margin	definition	value (v14)
road edge y_{margin}	lane wall singular where body edge $+ y_{margin}$ touches the road edge	5.4 mm (trained; search range 1 mm–1 m)
heading half-extent	$h(\theta)$ instead of $W/2$	+0 to +0.56 m
obstacle padding	effective size used in k' > the SUMO size	see table below
wall continuation	below $d_0 = 1$ cm the $1/d^2$ lane wall continues as a steep straight line	still pushes back after a step overshoots
k' floor	near the obstacle wall $(1 - l/\|\Delta x\|) \ge 10^{-4}$; inside contact it is set to 0.1	$U_{obs} \le 10^4\,\lambda_{obs}/\sqrt{\cdots}$, finite
τ floor	$\|\tau_x\|, \|\tau_y\| \ge 10^{-3}$	no division by 0
sensing window	obstacles counted only within $[-20, +r_{ahead}]$ m and $\|\Delta y\| \le 8$ m	$r_{ahead} = 97.5$ m (base), 10–250 m (adaptive)
gradient step	central difference, $\epsilon = 0.05$ m	–

Obstacle padding (`obstacle_effective_dimensions`). Extra clearance at which the k' wall sits beyond actual body contact, per side:

type	SUMO size $W \times L$	effective $w_{eff} \times l_{eff}$	lateral margin	longitudinal margin
bike	0.6×1.8	0.8×2.0	0.10 m	0.10 m
auto	1.4×2.6	1.6×3.0	0.10 m	0.20 m
car	2.0×5.0	2.1×5.0	0.05 m	0
bus, truck	2.5×12.0	2.7×12.0	0.10 m	0

Lane wall position. Driving straight, the ego centre is confined to $|y| < 5.25 - 0.874 - 0.0054 = 4.37$ m by the walls (lane centres at $0, \pm 3.5$ m). The wall strength $\delta_{lane} = 100$ sits at the upper end of its search range.

Hard limits behind the margins. The margins are soft (a strong enough pull can cross them within one 0.1 s step). What is enforced:

- Road: any part of the body off the road ⇒ collision (position is never clamped back).
- Vehicles: SUMO's own collision detector on the actual sizes.

4.4 Limits on the adapted parameters

The adaptive policy changes four potential parameters, each kept within three nested limits:

1. **Relative**: $|z| \le \ln 10 \Rightarrow p \in [\,0.1\,p_0,\ 10\,p_0\,]$.
2. **Absolute**: clamp to the `v14/base` search range.

3. **Rate**: smoothing with $T = 1$ s moves z at most $\frac{\Delta t}{T} \cdot 2 \ln 10 = 0.46$ per step $\Rightarrow$ a parameter changes by at most $\times 1.58$ (or $\div 1.58$) per 0.1 s.

parameter	p_0 (base)	$\times 0.1 \cdots \times 10$	absolute limits	effective range
λ_{obs}	190.7	19.1 – 1907	$0.01 - 10^4$	19.1 – 1907
τ_y	0.378	0.038 – 3.78	0.01 – 20	0.038 – 3.78
α	0.0209	0.0021 – 0.209	0.01 – 100	**0.01** – 0.209
r_{ahead} (m)	97.5	9.75 – 975	10 – 250	**10 – 250**

- Bold: the absolute limit is the binding one. α is pinned at its 0.01 floor in practice.
- Sign constraint: the weights from `ttc_front`, `ttc_side_front`, `drac_front` to λ_{obs} are ≥ 0 (enforced by reflection, $w \leftarrow |w|$).
- Fixed, not adapted: s_x, δ_{lane}, y_{margin}, $\tau_x = 1$, $r_{behind} = 20$ m, lateral band 8 m, the motion limits above. The policy cannot loosen any motion limit.

5 Identical SUMO scenarios and futures (v14)

All v14 comparisons (base vs adaptive, the numerator variants) rely on every candidate meeting **exactly the same traffic**. Three pieces make this possible: saved scenarios, futures, and repeatable SUMO runs.

5.1 Scenarios: saved starting states

- Created once by `step1_make_scenarios.py`: traffic runs for 30 s, the car nearest $x = 100$ m becomes the ego, and the full SUMO state is saved at that moment.
- Per scenario `scenarios/sNNNNN/`: `state.xml` (all vehicles, routes, RNG state), `ego.json` (the controller-held ego state v_x, v_y, heading), `meta.json` (seed, road length, start x).
- Saved **before** the controller acts, so a scenario does not depend on the DIPF parameters or the potential variant.
- All v14 sub-folders share one scenario folder, `v14/base/scenarios` (their `config.yaml` points `scenarios_dir` to it).
- Seed splits: train 0–99, validation 400–499, test 5000–5299 (never used for training or selection).
- Guard: `meta.json` records the road length; every step refuses scenarios made on another road.

5.2 Futures: several random continuations of one scenario

- A scenario fixes the **start**; the traffic after it is random (driver imperfection, speed deviation, lane-change decisions).
- Each scenario is reloaded with SUMO seeds $1, \ldots, n$ (`-seed`): **future** k = scenario + seed k. Same start, different random continuation.
- Futures per scenario: train 2, validation 3, test 3 $\Rightarrow$ one result is a success *rate*, not a single lucky or unlucky run.

- **Common random numbers**: every candidate (parameter set or policy) is run on the same scenarios × futures, so differences come from the controller, not from the traffic. Results are compared run by run (paired McNemar test).

```
cmd = [sumo, "-n", NET_FILE.name, "--load-state", "scenarios/sNNNNN/state.xml",
       "--step-length", "0.1", "--lateral-resolution", "0.25", ...]
cmd += ["--seed", str(future)]                        # future 1, 2, 3, ...
```

5.3 Repeatable runs: same scenario + same future ⇒ same result

Problem 1: address-space randomisation (ASLR). SUMO orders some internal work by memory address. Linux randomises memory addresses on every launch, so the same run (state, seed, commands) drew its random numbers in a different order each time: after one step two vehicles 300 m apart had swapped their speed perturbations, and an identical run could end in success once and in a collision the next time.

10 repeats of the same run	default (ASLR on)	v14 (ASLR off)
s00007, future 2	2 different outcomes	1
s00003, future 1	4 (2 successes, 2 collisions)	1
s00012, future 2	2 different outcomes	1

Fix: every SUMO process is started with address randomisation disabled (equivalent of `setarch -R`):

```
def _kill_with_parent():               # preexec_fn: runs just before SUMO starts
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    libc.prctl(1, signal.SIGKILL)                                  # SUMO dies with its parent
    ADDR_NO_RANDOMIZE = 0x0040000                                  # no ASLR:
    libc.personality(libc.personality(0xFFFFFFFF) | ADDR_NO_RANDOMIZE)
```

Problem 2: the result depended on the folder. SUMO's memory layout also depends on the length of its command line and environment. The same parameters run from `v14/base` and `v14/adaptive` gave different outcomes (network path `.../base/...` vs `.../adaptive/...`, and the `PWD` variable). Fix: identical command line and environment from every folder:

```
env = {k: v for k, v in os.environ.items() if k not in ("PWD", "OLDPWD", "_")}
# relative "-n" path, run in the network's folder, fixed environment
proc = subprocess.Popen(cmd + ["--remote-port", str(port)], preexec_fn=_kill_with_parent,
                        cwd=str(NET_FILE.parent), env=env)
```

Checks.

- Repeating a run gives the identical outcome and progress (30 of 30 repeats).
- The same parameters give identical results from `v14/base` and `v14/adaptive`.
- The adaptive policy with all weights 0 reproduces the trained fixed set run by run, and `adaptive/step3` reproduces `base/step3` exactly (success 0.646, progress 0.780).
- All runs must use the same SUMO binary: the scenarios were made with SUMO 1.27.1 (venv); SUMO 1.18 (`/usr/share/sumo`) cannot load them. `sumolib.checkBinary` follows `SUMO_HOME`.